Agentic LLM Ranking versus Trained Sequential Models: An Audited Study of ARAG, GRU Fusion and Text-Aware Sequence Modelling on Amazon Reviews 2023 (Video Games)

[Author Name]
[Course / Programme — Deep Learning Project]
[Institution]

October 2026

Abstract

Large language models (LLMs) can rank recommendation candidates by reasoning over a user's history in natural language. ARAG [12] organises this reasoning as a pipeline of four cooperating agents. We implement an ARAG-style recommender on the Amazon Reviews 2023 Video Games category [6] and ask whether a *trained* deep-learning component — a GRU sequential model [4] — improves it when late-fused into the LLM ranking. To isolate ranking from retrieval, every system ranks the same frozen 20-item candidate list per user (the held-out next item plus 19 popularity-matched negatives), and all contrasts are tested with a paired bootstrap. With `gpt-4o-mini`, ARAG reaches NDCG@5 = 0.424 on 100 test users and significantly beats the paper's LLM baselines; the paper's NLI+CSA ablation also reproduces significantly. However, properly trained sequential models — a comparison the paper does not make — are more accurate: on the same users a GRU reaches 0.520, SASRec [10] 0.536 (ARAG − SASRec = −0.112, 95% CI $[-0.213, -0.009]$), and on all 94,762 test users our *text-aware GRU*, whose item vectors add a projected frozen MiniLM embedding of the item's metadata, is best with 0.530. Fusing the GRU into ARAG significantly improves ARAG (H1: +0.098, $[+0.013, +0.182]$) but not the GRU (H2: +0.004, n.s.). ARAG's distinctive advantage is not accuracy but a much lower popularity bias and higher catalogue coverage. We document two errors found during the project — a degenerate candidate construction and a GRU weight-initialisation bug — each of which had reversed a headline conclusion, and we release an audit pipeline that re-derives every reported number from raw predictions. All post-correction experiments required zero new LLM calls.

Keywords: recommender systems; large language models; agentic retrieval-augmented generation; sequential recommendation; GRU4Rec; late fusion; evaluation methodology.

Contents

1 Introduction **3**

2 Related Work **4**

3 Data **4**

4 Evaluation Protocol **5**
4.1 Frozen candidate lists . . . 5
4.2 Users . . . 5
4.3 Metrics . . . 5
4.4 Statistical testing . . . 5

5 Methods **6**
5.1 System overview . . . 6
5.2 ARAG agent pipeline . . . 6
5.3 GRU sequential model . . . 7
5.4 Text-aware GRU . . . 7
5.5 SASRec . . . 7
5.6 Non-learned baselines . . . 8
5.7 Late fusion . . . 8

6 Experimental Setup **8**

7 Results **9**
7.1 Main comparison . . . 9
7.2 RQ1: does fusing the GRU help ARAG? . . . 9
7.3 RQ2: do the paper's claims hold? . . . 12
7.4 RQ3: trained sequential models . . . 12
7.5 History length . . . 14
7.6 Beyond accuracy . . . 16

8 Discussion **17**
8.1 What the base paper did and did not show . . . 17
8.2 Why fusion helps ARAG but not the GRU . . . 17
8.3 What text adds to a sequence model . . . 17
8.4 Why keep an LLM? . . . 17
8.5 Cost–quality trade-off . . . 18
8.6 A methodological lesson: candidate construction can manufacture conclusions . . . 18

9 Threats to Validity **18**

10 Conclusion and Future Work **19**

A Worked Example **20**

B Errata: the Superseded Run **20**

C Errata: the GRU Initialisation Bug **21**

D Reproducibility **21**

1 Introduction

Sequential recommendation predicts the next item a user will interact with from the order of their past interactions. For a decade the standard tools have been neural sequence models trained on interaction logs, from recurrent networks such as GRU4Rec [4] to self-attentive models such as SASRec [10]. Large language models offer a different route: they can read item titles and descriptions, infer a user's intent in natural language and rank candidates zero-shot [7]. Agentic pipelines push this further by splitting the task into specialised LLM calls. ARAG [12] uses a *User Understanding Agent*, a *Natural Language Inference* agent, a *Context Summary Agent* and an *Item Ranker Agent*, and reports large gains over single-prompt LLM baselines.

The two families have complementary strengths. Trained models capture collaborative co-occurrence patterns across hundreds of thousands of users, but see only opaque item IDs; LLMs understand item content but see only one user at a time and cost several API calls per recommendation. This project asks whether combining them helps, and what each contributes.

Research questions.

RQ1 On identical frozen candidate lists, does late-fusing a trained GRU into ARAG's ranking improve over ARAG alone (**H1**) and over the GRU alone (**H2**)?

RQ2 Do the base paper's claims — ARAG beats simpler LLM baselines, and each agent component contributes — hold on a new category and model?

RQ3 How accurate are trained sequential models (GRU, SASRec, and a text-aware GRU) on the same protocol, and how do they differ from ARAG beyond accuracy?

Contributions.

1. An ARAG-style four-agent recommender with strict JSON-schema outputs, a repairing ranking parser, and a disk cache that makes every LLM run exactly replayable at zero cost.
2. A ranking-isolating evaluation protocol: frozen 20-item lists with popularity-matched negatives, a single shared user sample for all systems, validation-only tuning and paired bootstrap confidence intervals.
3. For RQ1, fusion significantly improves ARAG (H1) but not the GRU (H2); for RQ2, a statistically significant reproduction of the base paper's baseline ordering and NLI+CSA ablation.
4. Strong trained baselines the paper omits: GRU4Rec, SASRec and a *text-aware GRU* that adds frozen sentence embeddings of item metadata to the item representation. All three are more accurate than ARAG at zero per-request cost; ARAG has far lower popularity bias (RQ3).
5. Two documented methodological lessons: a candidate-list construction that let popularity-like systems win, and an embedding-initialisation bug that crippled the GRU. Each reversed a headline conclusion; we report the diagnosis, the fix and both sets of numbers.

The rest of the report is organised as follows. Section 2 reviews related work, Section 3 describes the data and Section 4 the evaluation protocol. Section 5 presents the systems, Section 6 the experimental setup and Section 7 the results. Section 8 discusses them, Section 9 lists threats to validity and Section 10 concludes.

2 Related Work

Sequential recommendation. GRU4Rec [4] introduced recurrent networks [1] for session-based recommendation, trained with a next-item objective. SASRec [10] replaced recurrence with causal self-attention and is today a standard strong baseline. Both models represent items by learned ID embeddings, so items with few interactions receive poorly trained vectors — a weakness that becomes acute when user histories are short.

Text-based and transferable sequence models. Several works replace or augment ID embeddings with representations of item text. UniSRec [5] learns universal item representations from text with a pre-trained language model so that sequence models transfer across domains. Our text-aware GRU follows the same principle in its simplest form: a frozen sentence encoder [13, 15] supplies item features that a learned linear projection maps into the GRU's embedding space, added to the ID embedding.

LLMs as recommenders. LLMs can act as zero-shot rankers when given a user's history and a candidate list in the prompt [7]. Retrieval-augmented variants select which parts of the history to show. ARAG [12] decomposes ranking into user understanding, candidate–user natural language inference, context summarisation and final ranking, and reports gains of 25–42% NDCG@5 over recency-based and vanilla RAG prompting on three Amazon categories.

Evaluation methodology. Sampled-candidate evaluation is cheap but can distort system rankings, depending on how the negatives are drawn [11]. Strong, well-tuned simple baselines often match complex neural recommenders [2], which motivates our inclusion of content-similarity and item-based collaborative filtering [14] baselines. We use the paired bootstrap [3] for significance, with NDCG [8] as the primary metric.

3 Data

We use the Amazon Reviews 2023 corpus [6], Video Games category, in its official rating-only 5-core leave-last-out benchmark split. For each user, the last interaction is the test target, the second-to-last is the validation target and the remainder is training history. Each interaction is a user reviewing an item, the standard stand-in for a purchase in this corpus (purchase logs are not public). Item text comes only from product metadata (title, main category, categories, features and description, truncated to 120 words); review text is never used. Table 1 summarises the data.

Table 1: Dataset statistics (Amazon Reviews 2023, Video Games, 5-core leave-last-out).

Users	94,762
Interactions (train / valid / test)	625,062 / 94,762 / 94,762
Catalogue items with metadata	137,269
Items with at least one interaction	25,612 (18.7%)
Items in the training vocabulary	25,527
Training items per user: mean / median / 75th percentile	6.6 / 4 / 7
Raw files verified by SHA-256	4

Figure 1 shows that user histories are very short: 63.5% of users have only three to five training interactions. Perhaps surprisingly, this barely affects the trained models (Section 7.5).

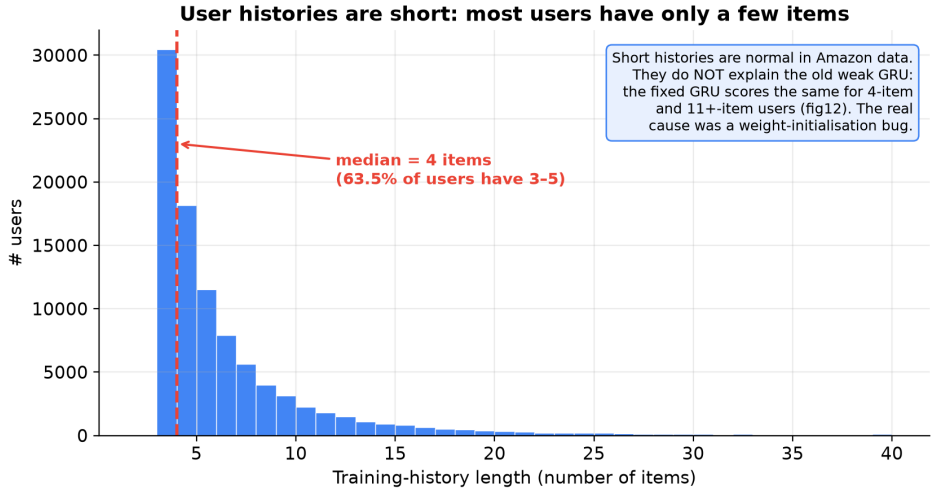


Figure 1: Distribution of training-history length per user. The median user has four training interactions.

4 Evaluation Protocol

4.1 Frozen candidate lists

For each validation and test user we build one 20-item candidate list: the true held-out target plus 19 negatives sampled without replacement (seed 42) from the 25,527 training-vocabulary items, with probability proportional to training interaction count, excluding everything the user interacted with in any split. The list is shuffled, hashed and frozen before any system is evaluated. Every system ranks exactly this list.

This design isolates *ranking* from *retrieval*: if each system retrieved its own candidates, a retrieval miss would score zero and the comparison would conflate the two stages. Popularity-proportional sampling makes negatives roughly as popular as targets (on average 1.01× on validation and 1.40× on test), so global popularity carries no free signal. A uniformly random permutation has expected Hit@5 of exactly 0.25, a useful sanity anchor.

4.2 Users

LLM systems are expensive, so all systems are compared on the *same* 100 randomly sampled test users (and 100 validation users for tuning). Non-LLM systems are additionally evaluated on all 94,762 test users, which gives near-deterministic estimates for them.

4.3 Metrics

With a single relevant item per user at rank $r \in \{1, \ldots, 20\}$,

$$\text{Hit@}K = \mathbb{1}[r \le K], \qquad \text{NDCG@}K = \frac{\mathbb{1}[r \le K]}{\log_2(r+1)}, \qquad \text{MRR} = \frac{1}{r}, \tag{1}$$

averaged over users. Hit@5 and NDCG@5 are primary; we also report Hit@10, NDCG@10 and MRR.

4.4 Statistical testing

For two systems A and B evaluated on the same users we resample users with replacement 10,000 times and report the mean per-user difference with its 95% percentile interval [3]. A contrast is called significant when the interval excludes zero. All hyperparameters are tuned on validation users only; the test set is never used for any selection.

5 Methods

5.1 System overview

Figure 2 shows the overall architecture. A data layer produces sequences and frozen lists; three families of rankers score each list; a fusion layer optionally blends an LLM ranking with a trained-model ranking; and an evaluation layer computes metrics and bootstrap intervals from the saved per-user rankings.

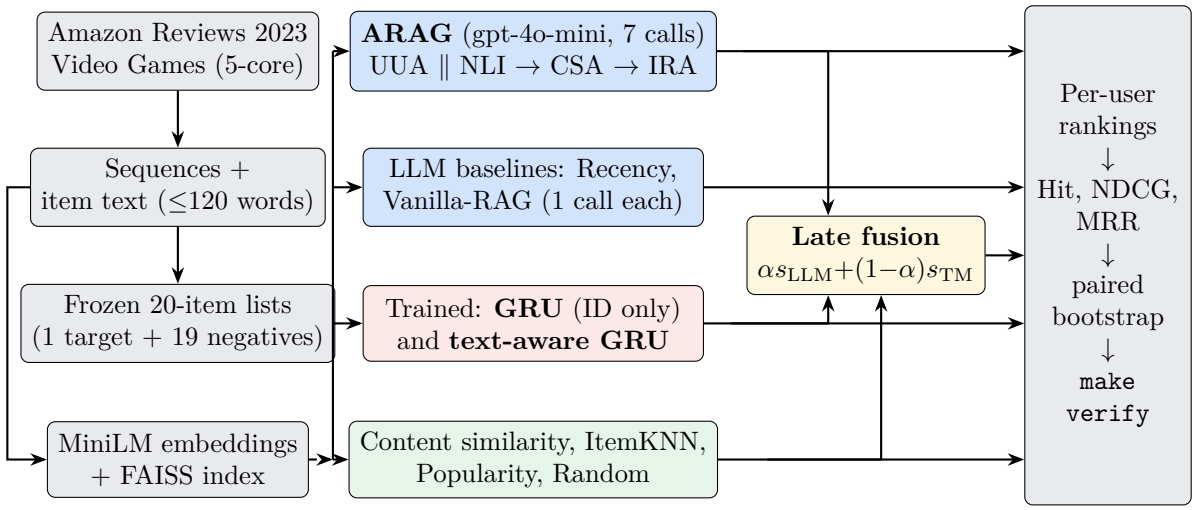


Figure 2: System architecture. All rankers score the same frozen list. The fusion layer blends the ARAG ranking with a trained-model or content ranking (s_{TM}); MiniLM embeddings feed the content baseline and, as item features, the text-aware GRU.

5.2 ARAG agent pipeline

Figure 3 details the agentic ranker. Each user's history (training items plus, for test, the validation item) is rendered as lines of the form *title* | *category* | *rating* | *date*. The long-term context C_{lt} holds up to 50 items and the session context C_{st} the last five.

- **UUA** (User Understanding Agent, 1 call): from C_{lt} and C_{st}, produces interests, current intent, dislikes and a summary of at most 120 words, S_{user}.
- **NLI** (4 calls of 5 candidates): for every candidate, a label (*support*/*neutral*/*contradict*), a compatibility score in $[0, 10]$ and a rationale, judged against the raw history.
- **CSA** (Context Summary Agent, 1 call): summarises the accepted candidates $\mathcal{I}^{+} = \{i : \mathrm{score}_i \geq \theta\}$ in at most 200 words, S_{ctx}. If no candidate passes θ, the top three by score are used and the fallback is counted.
- **IRA** (Item Ranker Agent, 1 call): given S_{user}, S_{ctx} and all 20 candidates, outputs a full permutation π_{LLM} with reasons.

UUA and NLI run in parallel. All outputs are requested as strict JSON schemas and validated. A ranking parser repairs malformed permutations (drops unknown IDs, removes duplicates, appends missing IDs by NLI score) and counts every repair. Two ablations follow the paper: *ARAG w/o NLI* (CSA summarises all 20 candidates) and *ARAG w/o NLI & CSA* (IRA receives only S_{user} and the raw candidates).

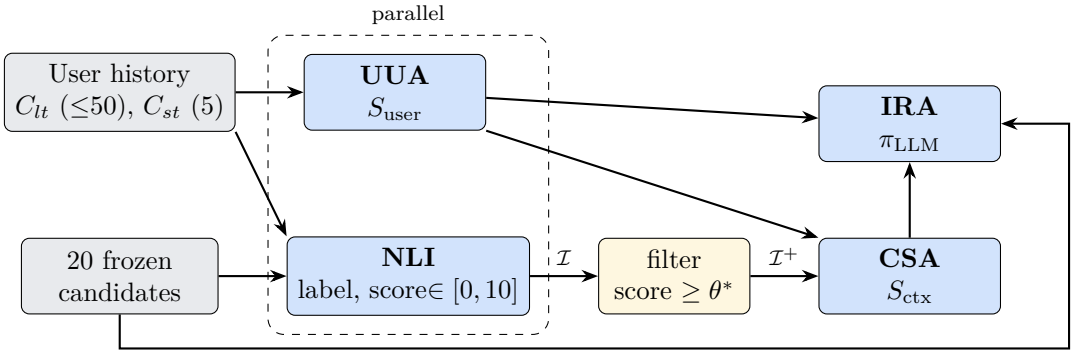


Figure 3: ARAG agent pipeline (seven LLM calls per user). $\theta^* = 5$ is tuned on validation.

5.3 GRU sequential model

Our trained model is a GRU4Rec-style network [4]. Let $\mathcal{V}$ be the training vocabulary (index 0 = padding, 1 = out-of-vocabulary). For a history $(i_1, \ldots, i_T)$, truncated to the last $T \leq 50$ items,

$$\mathbf{x}_t = \text{Dropout}(\mathbf{e}(i_t)), \qquad \mathbf{h}_t = \text{GRU}(\mathbf{x}_t, \mathbf{h}_{t-1}), \qquad \mathbf{u}_t = \mathbf{P}\,\text{Dropout}(\mathbf{h}_t), \tag{2}$$

with $\mathbf{e}(i) \in \mathbb{R}^{64}$, $\mathbf{h}_t \in \mathbb{R}^{d_h}$ and a projection $\mathbf{P} \in \mathbb{R}^{64 \times d_h}$ ($d_h = 64$ for the ID-only GRU, 128 for the text-aware GRU). Input and output embeddings are tied: the score of item j after step t is $\mathbf{u}_t^\top \mathbf{e}(j)$. The model is trained with full-softmax cross-entropy for next-item prediction at every prefix,

$$\mathcal{L} = -\sum_{u} \sum_{t=1}^{T_u - 1} \log \frac{\exp(\mathbf{u}_t^\top \mathbf{e}(i_{t+1}))}{\sum_{j \in \mathcal{V}} \exp(\mathbf{u}_t^\top \mathbf{e}(j))}. \tag{3}$$

At inference, the user vector is the final projected state $\mathbf{u}_T$ and candidates are ranked by $\mathbf{u}_T^\top \mathbf{e}(c)$. Item embeddings are initialised from $\mathcal{N}(0, 0.02^2)$ with the padding row at zero. This matters with tied embeddings: the default $\mathcal{N}(0, 1)$ initialisation makes the initial logits dot products of two unit-variance 64-d vectors ($|\mathbf{u}^\top \mathbf{e}| \approx 8$), so the softmax over 25,527 items is saturated from the first step and training stalls (Appendix C).

5.4 Text-aware GRU

Most item IDs are seen in few sequences, so their embeddings are estimated from little data. The text-aware GRU (Fig. 4) keeps the network of Section 5.3 unchanged and replaces only the item representation:

$$\tilde{\mathbf{e}}(i) = \mathbf{e}(i) + \mathbf{W}\,\mathbf{t}_i, \qquad \mathbf{t}_i = \text{MiniLM}(\text{text}_i) \in \mathbb{R}^{384}, \quad \mathbf{W} \in \mathbb{R}^{64 \times 384}, \tag{4}$$

where $\mathbf{t}_i$ is the frozen, L2-normalised `all-MiniLM-L6-v2` embedding [13, 15] of the item's metadata, and $\mathbf{W}$ is learned. The same $\tilde{\mathbf{e}}$ is used for inputs and for the tied output layer, so two items that never co-occur can still be close because their descriptions are similar. Text features are available for 25,526 of the 25,527 vocabulary items; padding and OOV rows are zero.

5.5 SASRec

As a second, independent trained baseline we use SASRec [10]: item embeddings plus learned position embeddings, two pre-norm Transformer blocks with two heads and a causal mask, and a final layer norm; the user vector is the output at the last real position. It shares the GRU's vocabulary, training sequences, tied full-softmax loss, optimiser, early stopping, initialisation and seeds, so the two trained models differ only in the sequence encoder.

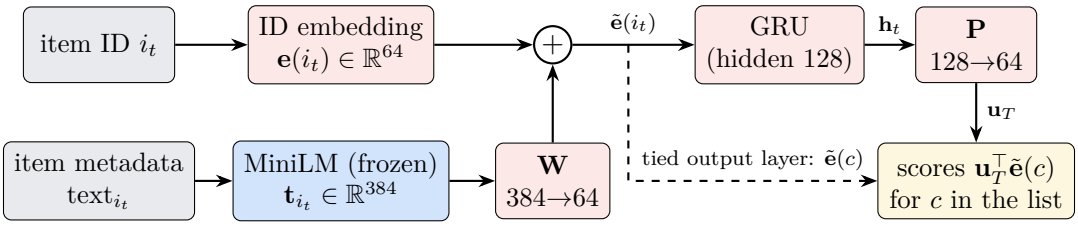


Figure 4: Text-aware GRU. The only change from the ID-only GRU is the item vector $\tilde{\mathbf{e}} = \mathbf{e} + \mathbf{W}\mathbf{t}$, shared by the input and the tied output layer.

5.6 Non-learned baselines

Content similarity. The user profile is a recency-weighted sum of the history's MiniLM vectors, $\mathbf{p}_u = \sum_{k=1}^{T} w_k \, \mathbf{t}_{i_k}$ with $w_k = \exp(-d\,(T-k))$, and candidates are ranked by $\mathbf{p}_u^\top \mathbf{t}_c$.

ItemKNN. From the binary user–item training matrix $\mathbf{X}$ we compute cosine item–item similarity $\text{sim}(a,b) = \frac{(\mathbf{X}^\top \mathbf{X})_{ab}}{\sqrt{n_a n_b}}$ [14], with n_a the number of users of item a, and score candidate c as $\sum_k w_k \, \text{sim}(i_k, c)$.

Popularity and random. Candidates ordered by training interaction count, or by a seeded random permutation.

LLM baselines. Following the paper, *LLM-Recency* prompts the ranker with the last ten history items, and *LLM-Vanilla-RAG* with the ten history items most similar to the mean candidate embedding; each uses one LLM call.

5.7 Late fusion

An LLM ranker outputs a permutation, not scores, so we fuse rankings. With rank $r_s(i)$ of item i under system s on a list of $n = 20$,

$$s_s(i) = 1 - \frac{r_s(i) - 1}{n - 1}, \qquad f(i) = \alpha \, s_{\text{LLM}}(i) + (1 - \alpha) \, s_{\text{TM}}(i), \tag{5}$$

with ties broken by NLI score, then item ID. $\alpha = 1$ reproduces ARAG exactly and $\alpha = 0$ reproduces the trained model exactly, so the trained model's contribution is exactly ablatable. $\alpha \in \{0, 0.1, \ldots, 1\}$ is chosen on the 100 validation users. We fuse ARAG with the ID-only GRU (*ARAG + GRU*, the original proposal), with the text-aware GRU and with content similarity.

6 Experimental Setup

Implementation. Python 3.11 and PyTorch; `sentence-transformers all-MiniLM-L6-v2` (384-d, normalised) with a FAISS [9] inner-product index; FastAPI and React for an interactive demonstrator. The LLM is OpenAI `gpt-4o-mini` at temperature 0 with strict JSON-schema outputs. A local `qwen3:8b` model via Ollama is supported by the same client and used for the test suite.

Hyperparameters. Table 2 lists all settings. The ID-only GRU was selected from a grid hidden $\in \{64, 128\}\times$ dropout $\in \{0.2, 0.5\}$ (best: 64/0.2, validation NDCG@5 0.531); SASRec from dropout $\in \{0.2, 0.5\}$ (best 0.5, 0.530). The text-aware GRU uses hidden 128 and dropout 0.2 without a grid. All trained models use the same 30-epoch cap and select the checkpoint with the best validation NDCG@5 over all 94,762 validation users.

Table 2: Hyperparameters. All selections use validation data only.

Component	Setting	Value
Trained models	item dimension; init	64; $\mathcal{N}(0, 0.02^2)$
	optimiser; learning rate	Adam; 10^{-3}
	batch size; max sequence length	256; 50
	epochs (max); early-stopping patience	30; 3
	seeds	0, 1, 2
GRU / text-aware GRU	hidden dimension; dropout	64 / 128; 0.2
SASRec	blocks; heads; dropout	2; 2; 0.5
Text-aware GRU	text encoder (frozen); projection	MiniLM-L6-v2; 384→64
ARAG	NLI acceptance threshold θ^* (grid 4–7)	5
	candidates per NLI call; temperature	5; 0
Fusion	α^*: ARAG+GRU / +text-GRU / +content	0.2 / 0.1 / 0.4
Content, ItemKNN	recency decay d^* (grid 0, 0.3, 0.6, 1.0)	0.3 / 0.3
Lists	size; negatives; sampling seed	20; 19; 42
Bootstrap	resamples; interval	10,000; 95% percentile

Tuning. Validation NDCG@5 for the NLI threshold was $\theta = 4$: 0.4228, $\theta = 5$: 0.4305, $\theta = 6$: 0.3993, $\theta = 7$: 0.3962 , giving $\theta^* = 5$. Table 6 shows the validation α curves.

Cost and reproducibility. Every LLM response is stored in a disk cache keyed by the hash of (model, prompt, schema). The corrected evaluation made 2,263 new calls (about $1.06); total unique project spend across both protocol versions was 4,668 calls, 10.3 M input and 0.83 M output tokens, $2.04. With `REPLAY_ONLY=1`, any uncached call raises an error, so all later experiments (content baselines, text-aware GRU, the corrected GRU, SASRec and new fusions) were evaluated at a guaranteed cost of $0. A verification script recomputes every metric from the saved per-user rankings and checks dataset and list hashes against the run record.

Compute. All training ran on a laptop CPU (Apple silicon, 16 GB RAM). The ID-only GRU grid and seeds took about 2 h 25 min, the text-aware GRU about 9 min per seed and SASRec about 15 min per seed, after two memory optimisations (logits computed only at non-padded positions; chunked validation) that do not change the loss.

7 Results

7.1 Main comparison

Table 3 reports every system on the same 100 test users, and Table 4 the non-LLM systems on all 94,762 test users. Table 5 gives the paired contrasts.

Overview. ARAG is the best LLM-based system (NDCG@5 0.424): it significantly beats LLM-Recency (+0.139), LLM-Vanilla-RAG (+0.162) and popularity (+0.315). The trained sequential models are more accurate still: on the same users the GRU reaches 0.519 (seed 0), SASRec 0.536 and the fused ARAG + text-GRU 0.532. Figure 5 visualises the comparison and Fig. 6 the hypothesis tests.

7.2 RQ1: does fusing the GRU help ARAG?

H1 is supported. At the validation-optimal $\alpha^* = 0.2$, ARAG + GRU improves NDCG@5 over ARAG by +0.098 [+0.013, +0.182]. **H2 is not supported**: the fused system is no better than the GRU alone (+0.004 [−0.038, +0.045]). Figure 7 and Table 6 show the shape: validation and

Table 3: All systems on the same 100 sampled test users (best value per column in bold). Trained models are shown per seed. Standard deviations across users are about 0.4–0.5 for Hit and 0.3–0.45 for NDCG; see Table 5 for paired confidence intervals.

System	Hit@5	NDCG@5	Hit@10	NDCG@10	MRR
Non-learned					
Random	0.220	0.125	0.470	0.205	0.162
Popularity	0.200	0.109	0.470	0.197	0.150
Content similarity	0.590	0.439	0.820	0.510	0.429
ItemKNN	0.540	0.399	0.670	0.442	0.392
Trained sequential models					
GRU, ID only (seed 0)	0.680	0.519	0.850	0.575	0.499
GRU, ID only (seed 1)	**0.720**	0.526	0.840	0.563	0.487
GRU, ID only (seed 2)	0.690	0.515	0.850	0.565	0.486
Text-aware GRU (seed 0)	**0.720**	0.506	0.900	0.565	0.465
Text-aware GRU (seed 1)	0.700	0.497	0.900	0.562	0.463
Text-aware GRU (seed 2)	0.700	0.499	**0.910**	0.568	0.468
LLM baselines and ablations					
LLM-Recency	0.400	0.285	0.630	0.358	0.300
LLM-Vanilla-RAG	0.380	0.263	0.670	0.355	0.283
ARAG w/o NLI & CSA	0.470	0.362	0.630	0.414	0.372
ARAG w/o NLI	0.510	0.376	0.720	0.445	0.380
Agentic LLM and fusion					
ARAG	0.530	0.424	0.700	0.479	0.432
ARAG + GRU	0.690	0.523	0.840	0.574	**0.502**
ARAG + content	0.590	0.423	0.810	0.494	0.411
ARAG + text-aware GRU	**0.720**	**0.532**	**0.910**	**0.594**	0.501

Table 4: Non-LLM systems on all 94,762 test users (mean ± standard deviation over three seeds for the GRUs).

System	Hit@5	NDCG@5	MRR
Random	0.252	0.149	0.181
Popularity	0.213	0.128	0.166
ItemKNN	0.519	0.406	0.406
Content similarity	0.569	0.411	0.402
GRU, ID only (3 seeds)	0.686 ± 0.001	0.505 ± 0.001	0.478 ± 0.000
Text-aware GRU (3 seeds)	**0.716 ± 0.001**	**0.530 ± 0.001**	**0.500 ± 0.002**

Table 5: Paired bootstrap contrasts on the same 100 test users (ΔNDCG@5, 95% CI, 10,000 resamples). GRU and text-aware GRU refer to seed 0 unless stated. H1′/H2′ repeat H1/H2 with the text-aware GRU.

Contrast	ΔNDCG@5	95% CI	Sig.
H1: ARAG + GRU − ARAG	+0.098	[+0.013, +0.182]	yes
H2: ARAG + GRU − GRU	+0.004	[−0.038, +0.045]	no
ARAG − LLM-Recency	+0.139	[+0.063, +0.217]	yes
ARAG − LLM-Vanilla-RAG	+0.162	[+0.079, +0.247]	yes
ARAG − ARAG w/o NLI	+0.049	[−0.011, +0.111]	no
ARAG − ARAG w/o NLI & CSA	+0.062	[+0.016, +0.113]	yes
ARAG − GRU (seed 1)	−0.101	[−0.201, +0.001]	no
ARAG − Popularity	+0.315	[+0.224, +0.408]	yes
Text-aware GRU − GRU	−0.013	[−0.064, +0.039]	no
Text-aware GRU − ARAG	+0.081	[−0.016, +0.179]	no
Content similarity − ARAG	+0.014	[−0.088, +0.116]	no
ItemKNN − ARAG	−0.025	[−0.132, +0.082]	no
H1′: ARAG + text-GRU − ARAG	+0.108	[+0.016, +0.197]	yes
H2′: ARAG + text-GRU − text-GRU	+0.026	[−0.001, +0.054]	no
ARAG + content − ARAG	−0.001	[−0.070, +0.066]	no

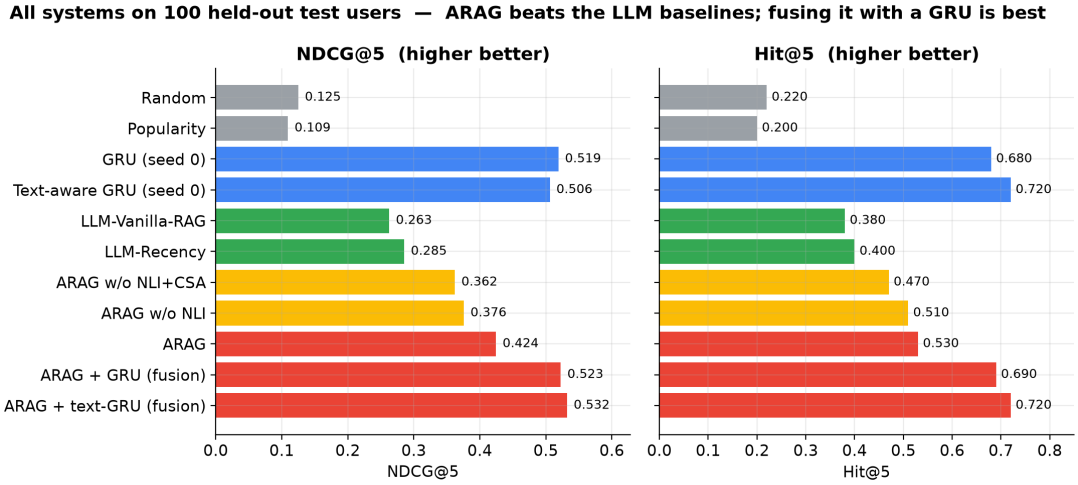


Figure 5: NDCG@5 and Hit@5 on the 100 test users.

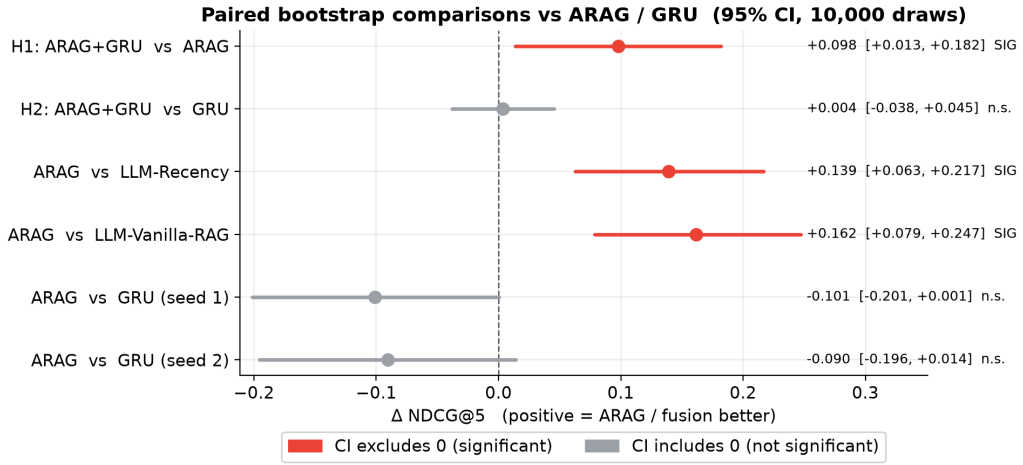


Figure 6: Forest plot of the key contrasts (ΔNDCG@5, 95% CI).

test both peak at small α (mostly GRU) and fall steadily towards pure ARAG. Adding a little of ARAG's ranking is harmless or slightly helpful; giving it most of the weight costs about 0.1 NDCG@5. With the text-aware GRU the picture is the same (H1′: +0.108, significant; H2′: +0.026, n.s. for seed 0, +0.031 [+0.002, +0.061] against the three-seed average).

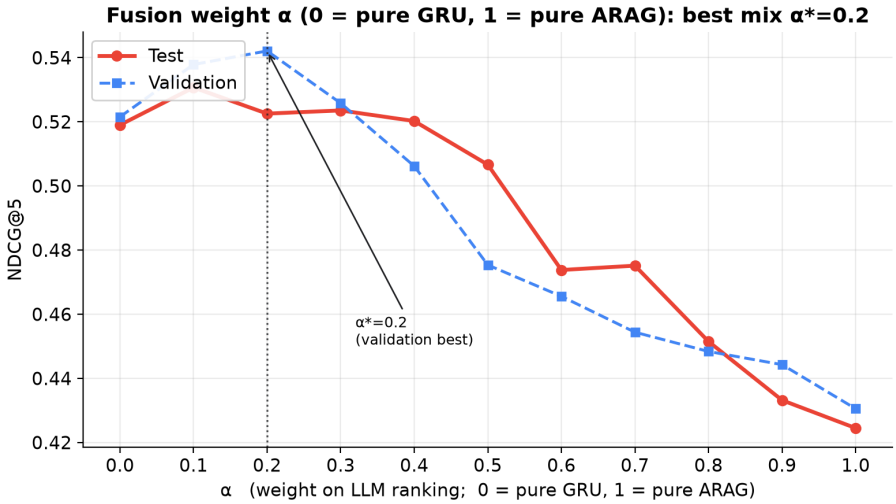


Figure 7: NDCG@5 of ARAG + GRU as a function of the fusion weight α (0 = GRU only, 1 = ARAG only).

Table 6: NDCG@5 against the fusion weight α (α^* in bold where it falls on a shown grid point; for ARAG + GRU, $\alpha^* = 0.2$; for ARAG + text-GRU, $\alpha^* = 0.1$). Validation curves are on the 100 validation users.

α	0.0	0.2	0.3	0.4	0.6	0.8	0.9	1.0
ARAG + GRU, validation	0.522	**0.542**	0.526	0.506	0.466	0.448	0.444	0.431
ARAG + GRU, test	0.519	0.523	0.524	0.520	0.474	0.451	0.433	0.424
ARAG + text-GRU, validation	0.557	0.551	0.538	0.522	0.463	0.453	0.440	0.431
ARAG + content, validation	0.441	0.451	0.465	**0.489**	0.455	0.451	0.438	0.431

As an exploratory, post-hoc analysis on the test users (not used for any selection), Fig. 8 compares the linear rank transform of Eq. (5) with reciprocal-rank fusion and an exponential transform. The best points (0.531–0.546) differ by less than the confidence intervals of Table 5.

7.3 RQ2: do the paper's claims hold?

Both of the paper's baseline orderings reproduce significantly (Table 5), and the ablation staircase has the same shape as in all three of the paper's categories (Fig. 9): removing NLI and CSA significantly hurts (-0.062 $[-0.113, -0.016]$), and removing NLI alone is directionally negative but not significant (-0.049 $[-0.111, +0.011]$). Figure 10 sets our numbers next to the paper's Table 1. The absolute values are not comparable (different dataset version, category, model and candidate protocol), but ARAG's relative lift over the best baseline (+48.8% NDCG@5) is at least as large as the paper's (+25.6% to +42.1%).

7.4 RQ3: trained sequential models

Table 4 and Table 7 report the trained models on all 94,762 test users. The ID-only GRU reaches NDCG@5 0.505 ± 0.001 and SASRec 0.508 (three seeds: 0.506, 0.510, 0.506), a negligible +0.003 difference; two different sequence encoders under one protocol land at the same accuracy. The

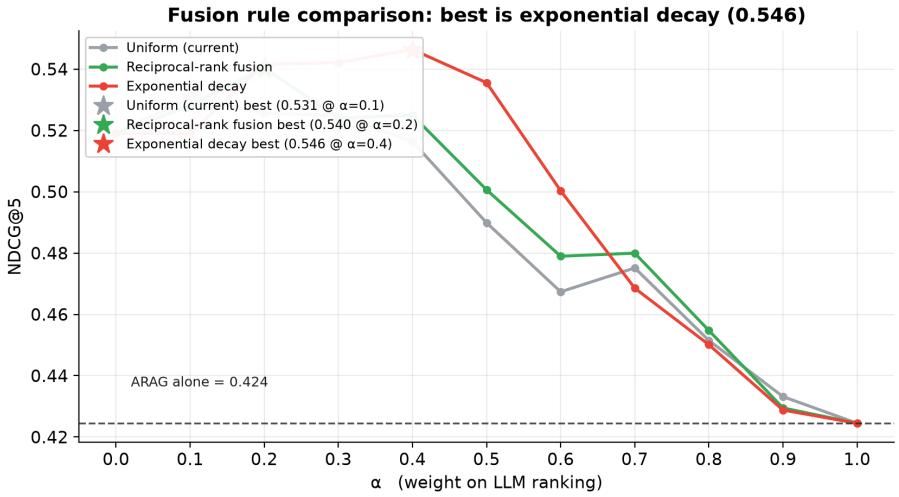


Figure 8: Exploratory comparison of rank-to-score transforms for ARAG + ID-only GRU on the test users. Shown for completeness; selecting a rule on these numbers would leak test data.

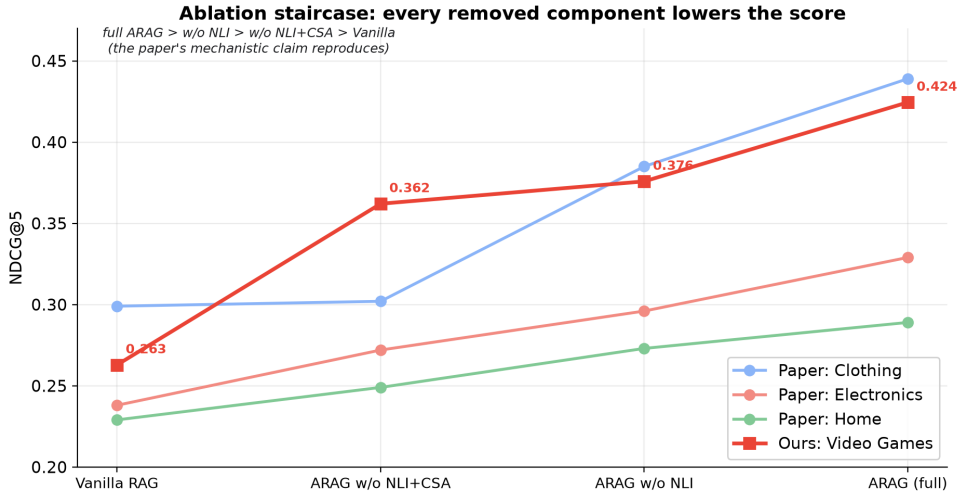


Figure 9: Ablation staircase: ours (Video Games) next to the paper's three categories.

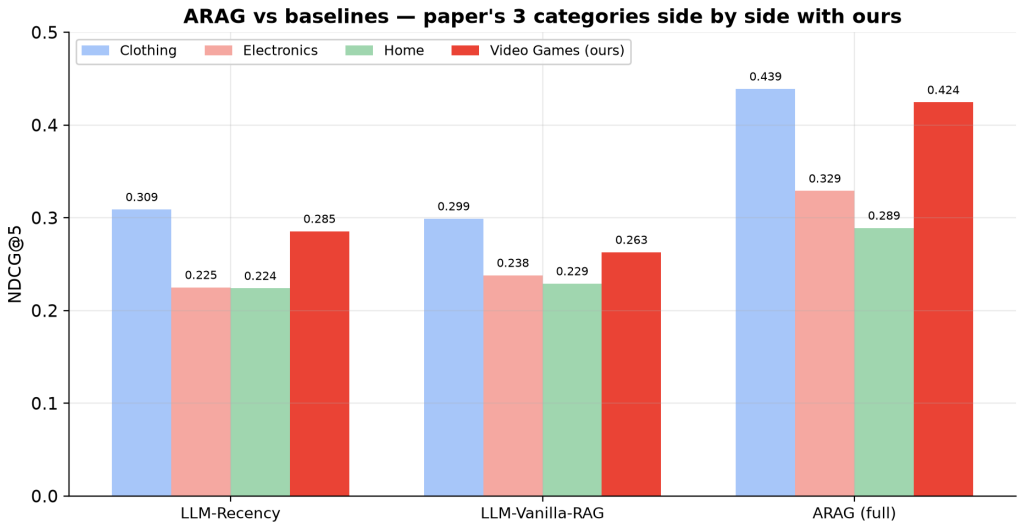


Figure 10: Our results next to the base paper's Table 1. Absolute values are shown for context only; the setups differ in dataset version, category, LLM and candidate construction.

text-aware GRU is best at 0.530 ± 0.001, $+0.026$ $[+0.024, +0.027]$ over the ID-only GRU and $+0.120$ $[+0.117, +0.122]$ over content similarity. Figure 12 shows that text mainly speeds up learning: the text-aware GRU passes 0.53 validation NDCG@5 by epoch 4, the ID-only GRU by epoch 28.

On the 100 LLM users the trained models are all above ARAG (Fig. 11): ARAG − SASRec $= -0.112$ $[-0.213, -0.009]$ (significant), ARAG − GRU $= -0.095$ $[-0.196, +0.006]$ and ARAG − text-aware GRU $= -0.076$ $[-0.173, +0.021]$ (three-seed averages, not significant at $n = 100$). Test-time sanity checks on SASRec confirm that it uses the user's own history: replacing it with another user's history drops NDCG@5 from 0.509 to 0.181, an empty history gives 0.130 (near random), and a shuffled history 0.434. Only 0.4% of targets and no negatives are out of vocabulary.

Table 7: Key paired contrasts involving SASRec and ARAG (ΔNDCG@5 = first − second, 95% CI; * = CI excludes 0). Seeded models are averaged over three seeds per user.

Contrast (ΔNDCG@5)	Users	95% CI
SASRec vs GRU (ID)	94,762	+0.003 [+0.002, +0.004]*
SASRec vs Text-aware GRU	94,762	−0.023 [−0.024, −0.021]*
SASRec vs ItemKNN	94,762	+0.102 [+0.100, +0.105]*
ARAG vs SASRec	100	−0.112 [−0.213, −0.009]*
ARAG vs GRU (ID)	100	−0.095 [−0.196, +0.006]
ARAG vs Text-aware GRU	100	−0.076 [−0.173, +0.021]
ARAG + text-GRU vs ARAG	100	+0.108 [+0.016, +0.197]*
ARAG + text-GRU vs SASRec	100	−0.004 [−0.052, +0.043]

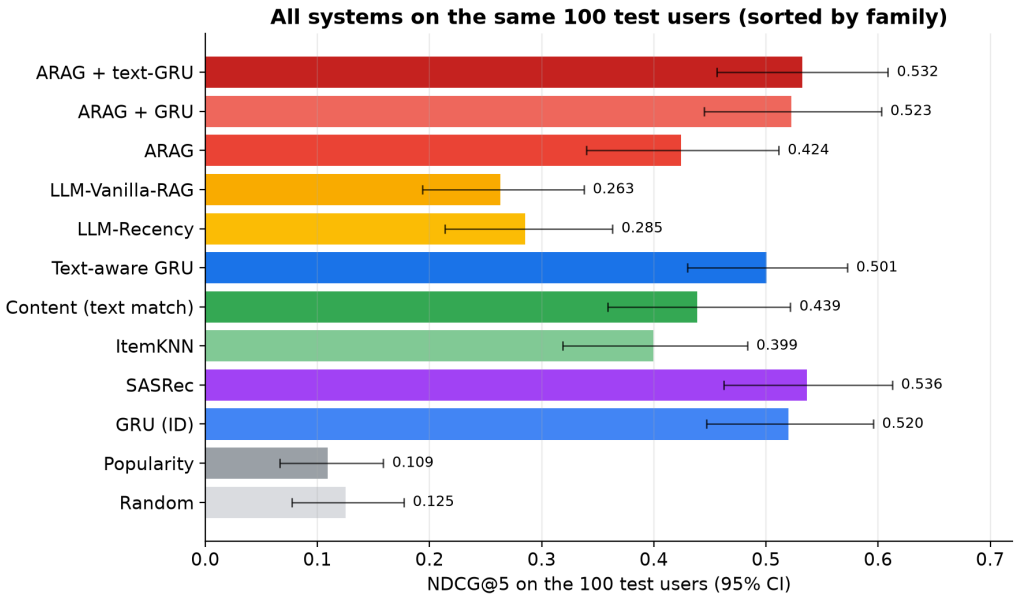


Figure 11: All systems on the 100 test users (seeded models averaged over three seeds), with 95% bootstrap confidence intervals.

7.5 History length

Table 8 splits all test users by the number of interactions before the test item. The trained models are nearly flat (GRU 0.497 for four items, 0.519 for eleven or more); the text-aware GRU leads the ID-only GRU in every bucket by 0.024–0.027 (all significant). Content similarity degrades with longer histories, whose averaged profile becomes less specific, while ItemKNN

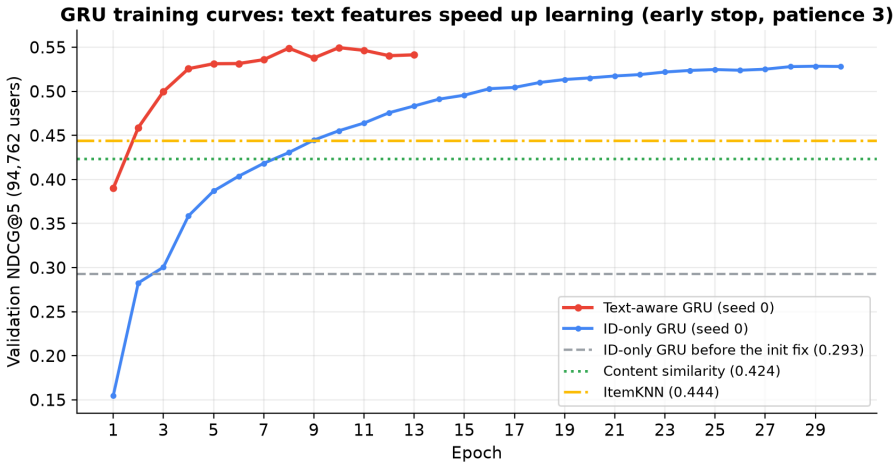


Figure 12: Validation NDCG@5 per epoch (seed 0) of the text-aware and ID-only GRUs, against the ID-only GRU before the initialisation fix and the two non-learned baselines.

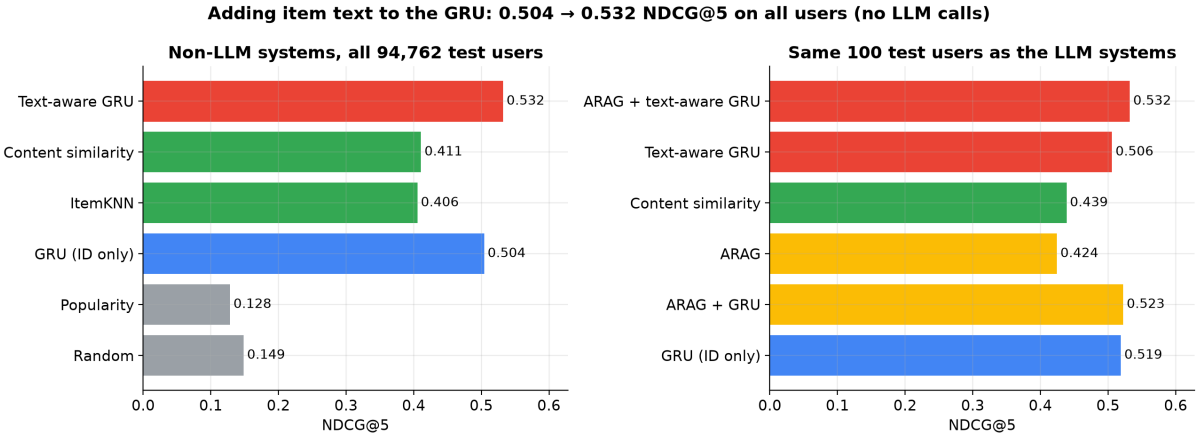


Figure 13: Left: non-LLM systems on all 94,762 test users. Right: the same 100 users used for the LLM systems.

improves. On the 100 LLM users (44/30/26 users per bucket, Fig. 14) ARAG is below the GRU in every bucket, none significantly.

Table 8: NDCG@5 by history length, all test users (seeded models averaged over three seeds).

History	Users	Content similarity	ItemKNN	GRU (ID only)	Text-aware GRU	SASRec
4 items	30,451	0.420	0.389	0.497	0.521	0.502
5–6	29,707	0.417	0.398	0.503	0.530	0.508
7–10	20,678	0.402	0.419	0.510	0.536	0.511
11+	13,926	0.391	0.437	0.519	0.543	0.513

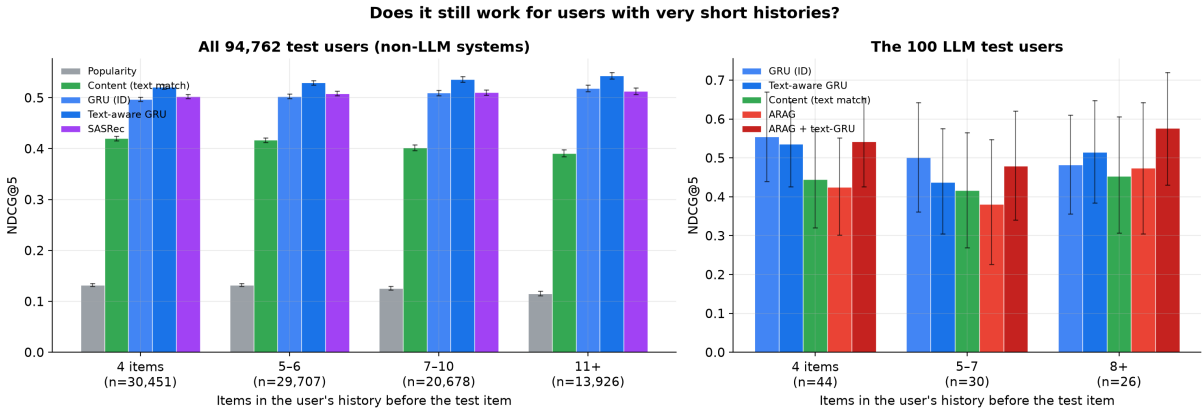


Figure 14: NDCG@5 by history length. Left: all test users. Right: the 100 LLM users.

7.6 Beyond accuracy

Table 9 reports, on the 100 users, the popularity lift of each system's top five (mean log(1 + train count) of the top five minus the 20-candidate average), intra-list diversity (one minus the mean pairwise MiniLM cosine of the top five) and coverage (distinct top-5 items over distinct candidates). ARAG's lift is 0.127, against 0.74–0.79 for the trained models and 0.98 for ItemKNN, and its coverage is the highest of the learned systems (0.275 vs. ≈ 0.25). Diversity is essentially equal (≈ 0.72) except for content similarity (0.646). Fusion at the chosen α inherits the trained model's popularity bias.

Table 9: Beyond-accuracy metrics on the 100 test users (seeded models averaged over three seeds).

System	NDCG@5	Popularity lift	Diversity (ILD)	Coverage
Popularity	0.109	1.783	0.723	0.181
ItemKNN	0.399	0.983	0.723	0.241
Content similarity	0.439	0.083	0.646	0.278
GRU (ID only)	0.520	0.768	0.721	0.252
Text-aware GRU	0.501	0.741	0.718	0.253
SASRec	0.536	0.787	0.718	0.247
LLM-Recency	0.285	0.162	0.724	0.281
LLM-Vanilla-RAG	0.263	0.278	0.725	0.273
ARAG	0.424	0.127	0.726	0.275
ARAG + GRU	0.523	0.728	0.720	0.254
ARAG + text-GRU	0.532	0.750	0.717	0.254

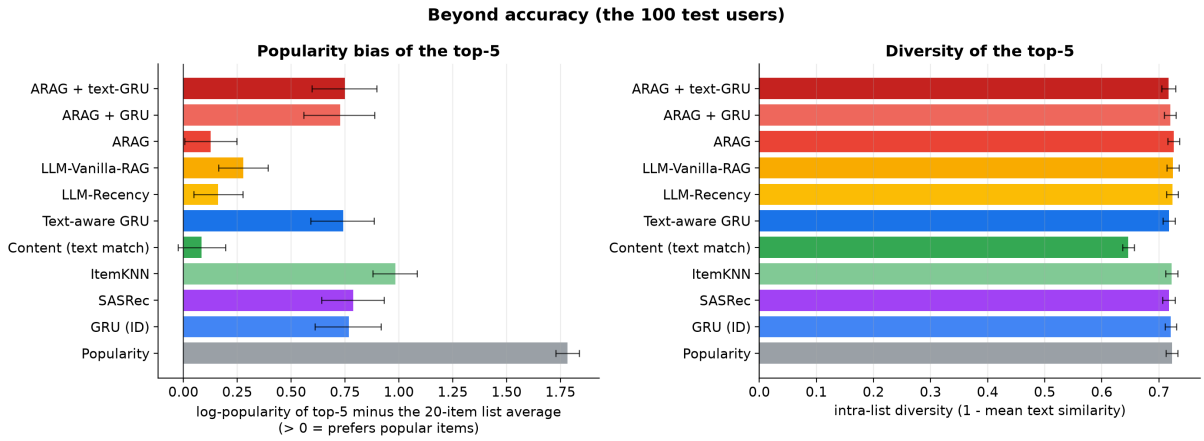


Figure 15: Popularity bias (left) and diversity (right) of each system's top five.

8 Discussion

8.1 What the base paper did and did not show

ARAG's claims are about LLM-based recommenders, and they hold: the agentic decomposition beats single-prompt baselines and its NLI+CSA components contribute. The paper does not report trained sequential baselines, and on our protocol those are more accurate than ARAG. Our results therefore do not contradict the paper; they show that its comparison was incomplete. They are also limited to one category, one LLM and 100 LLM-evaluated users. This pattern — carefully tuned standard baselines matching or beating newer methods — echoes Dacrema et al. [2].

8.2 Why fusion helps ARAG but not the GRU

The two rankers make different mistakes: ARAG judges candidates by their descriptions against a language summary of the user, the GRU by co-occurrence patterns across 625k interactions. Adding a small amount of ARAG's ranking to the GRU ($\alpha^* = 0.2$) recovers most of the GRU's accuracy and substantially improves ARAG, but the extra information is not enough to lift the fused system above the GRU at $n = 100$. Content similarity is statistically tied with ARAG, which suggests that much of ARAG's skill on this task comes from reading item text.

8.3 What text adds to a sequence model

The text-aware GRU changes only the item vector (Eq. (4)). Its gain over the correctly trained ID-only GRU is modest (+0.026) but consistent across seeds and history lengths, and it learns much faster (Fig. 12). Text features place rarely seen items near semantically similar ones, in line with transferable text-based sequence models [5]. An earlier version of this report attributed a far larger gain (+0.17) to text; that figure was an artefact of the initialisation bug (Appendix C).

8.4 Why keep an LLM?

ARAG has the lowest popularity bias of any learned system and the highest coverage, produces natural-language explanations for every recommendation, and handles free-text queries (used by our demonstrator's search). These properties are not measured by NDCG@5 but matter in a product.

8.5 Cost–quality trade-off

ARAG needs seven LLM calls per user (about $1 per 100-user evaluation with `gpt-4o-mini`, and seconds of latency per user). The text-aware GRU scores all 94,762 test users in seconds on a CPU at no marginal cost. On this protocol the trained models are the better operating point for accuracy; ARAG is the better operating point for explanation and low popularity bias.

8.6 A methodological lesson: candidate construction can manufacture conclusions

An earlier version of the study sampled negatives uniformly from all 137,260 catalogue items with text. Because 81% of those items have no interactions, a mean of 15.5 of the 19 negatives were unknown to popularity (all tied at zero) and to the GRU (all mapped to one OOV embedding). The effective task for those systems was a choice among about 4.5 items. Popularity reached Hit@5 = 0.97, the GRU NDCG@5 = 0.82, and fusion appeared to improve ARAG by +0.349 ("H1 supported"). The flaw was found in review; the lists were rebuilt with popularity-matched in-vocabulary negatives, every system was re-evaluated and the GRU retrained. The conclusion reversed (Fig. 16). The general point, also made by Krichene and Rendle [11], is that sampled-candidate protocols must be checked for shortcuts that some systems can exploit and others cannot. A second error — the GRU's embedding initialisation (Appendix C) — reversed the conclusion again; it was exposed only when an independent architecture (SASRec) showed the same symptom. A weak baseline should be a reason to check the baseline.

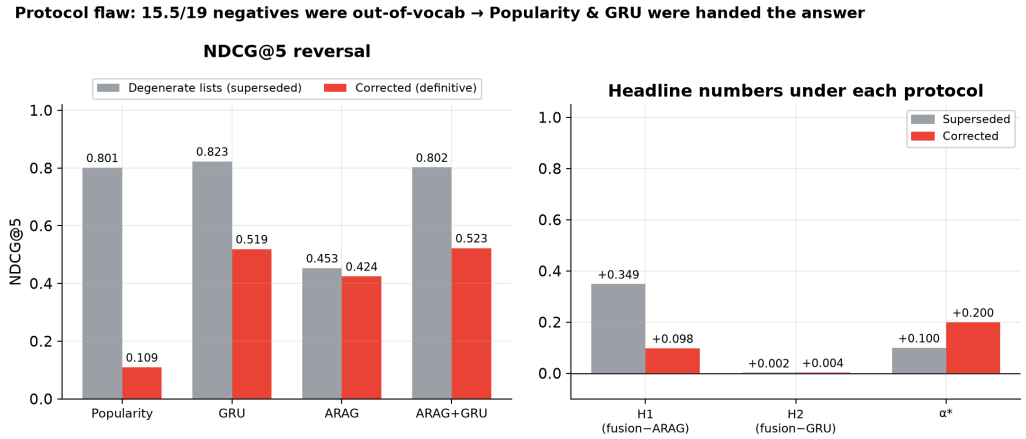


Figure 16: The original (degenerate) candidate lists versus the corrected lists (right panel: final numbers with the corrected GRU).

9 Threats to Validity

1. **Sample size.** Only 100 users are evaluated with the LLM (cost). Confidence intervals on NDCG@5 differences are roughly ±0.08–0.10, so the top systems (GRU, SASRec, text-aware GRU and the fusions) cannot be separated from each other, and ARAG versus the GRU is borderline. About 300 users (≈ $3) would be needed to resolve differences of ~ 0.05.

2. **Sampled negatives.** Popularity-matched negatives remove the popularity shortcut but are easier than candidates mined by a real retriever. Results describe ranking quality on hard 20-item lists, not end-to-end retrieval quality.

3. **Slight tilt of the negatives.** Popularity scores slightly below random (Hit@5 0.213 vs. 0.252 on all users), so negatives are on average a little more popular than targets. This

slightly disadvantages frequency-sensitive models, including the GRUs. The effect (~ 0.04 Hit@5) is much smaller than the gaps we interpret.

4. **Leave-last-out leakage.** Other users' later interactions can appear in the training data of the GRUs and ItemKNN. This is inherent to the official split and affects those systems equally; content similarity does not use other users' data.
5. **Training budget and search.** The ID-only GRU was still improving slowly at the 30-epoch cap and the hyperparameter grids are small; the trained models are, if anything, under-estimated.
6. **Tuning noise.** α and θ are tuned on 100 validation users; the validation and test α curves differ in detail near their maxima ($\alpha = 0.1$–0.3).
7. **Scope.** One category, one dataset, one LLM. The study is ARAG-inspired, not an exact reproduction: the paper does not publish prompts, its threshold or its candidate construction, and it used `gpt-3.5-turbo` on the 2016 Amazon data.
8. **Offline metrics only.** No online or human evaluation; one held-out item per user defines relevance.

10 Conclusion and Future Work

We built an ARAG-style agentic recommender and evaluated it against, and fused with, trained sequential models under a protocol that isolates ranking and tests every contrast statistically. First, the agentic pipeline significantly beats simpler LLM baselines and its NLI+CSA ablation, reproducing the base paper's claims on a new category and model. Second, properly trained sequential recommenders — GRU4Rec, SASRec and our text-aware GRU — are more accurate than ARAG (NDCG@5 0.505–0.530 on all test users; ARAG 0.424 on its 100 users, significantly below SASRec), at no per-request cost. Fusing the GRU into ARAG significantly improves ARAG (H1) but not the GRU (H2). Third, ARAG's distinctive value lies beyond accuracy: much lower popularity bias, higher coverage and natural-language explanations. Finally, both a candidate-construction flaw and an initialisation bug reversed our headline conclusion before they were found; evaluation pipelines and baselines must be audited, not just the proposed method.

Future work: a larger LLM-evaluated sample to separate the top systems; negatives mined by a retriever; SASRec with the text-aware item representation; review text as an item feature; feeding the trained model's scores to the IRA agent as an explicit feature instead of fusing afterwards; and replication on further categories and LLMs.

References

[1] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of EMNLP*, 2014.

[2] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach. Are we really making much progress? A worrying analysis of recent neural recommendation approaches. In *Proceedings of ACM RecSys*, 2019.

[3] B. Efron and R. J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

[4] B. Hidasi, A. Karatzoglou, L. Baltrunas, and D. Tikk. Session-based recommendations with recurrent neural networks. In *International Conference on Learning Representations (ICLR)*, 2016.

[5] Y. Hou, S. Mu, W. X. Zhao, Y. Li, B. Ding, and J.-R. Wen. Towards universal sequence representation learning for recommender systems. In *Proceedings of ACM SIGKDD*, 2022.

[6] Y. Hou, J. Li, Z. He, A. Yan, X. Chen, and J. McAuley. Bridging language and items for retrieval and recommendation. *arXiv preprint arXiv:2403.03952*, 2024.

[7] Y. Hou, J. Zhang, Z. Lin, H. Lu, R. Xie, J. McAuley, and W. X. Zhao. Large language models are zero-shot rankers for recommender systems. In *Proceedings of ECIR*, 2024.

[8] K. Järvelin and J. Kekäläinen. Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, 20(4):422–446, 2002.

[9] J. Johnson, M. Douze, and H. Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.

[10] W.-C. Kang and J. McAuley. Self-attentive sequential recommendation. In *Proceedings of IEEE ICDM*, 2018.

[11] W. Krichene and S. Rendle. On sampled metrics for item recommendation. In *Proceedings of ACM SIGKDD*, 2020.

[12] R. Y. Maragheh, P. Vadla, P. Gupta, K. Zhao, A. Inan, K. Yao, J. Xu, P. Kanumala, J. Cho, and S. Kumar. ARAG: Agentic retrieval augmented generation for personalized recommendation. *arXiv preprint arXiv:2506.21931*, 2025.

[13] N. Reimers and I. Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of EMNLP-IJCNLP*, 2019.

[14] B. Sarwar, G. Karypis, J. Konstan, and J. Riedl. Item-based collaborative filtering recommendation algorithms. In *Proceedings of WWW*, 2001.

[15] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

A Worked Example

A real test user (`AEDTQXNMWBEI7UECETV5JVUEWNDQ`) with a four-item history: *The Legend of Zelda: Breath of the Wild* (Wii U), *Need for Speed Most Wanted U* (Wii U), *Hot Wheels World's Best Driver* and *LEGO Marvel* (PC). The frozen list contains twenty plausible gaming products — for example *Demon's Souls* (PS3), *New Super Mario Bros. U*, a Luigi amiibo, *Ghost of Tsushima* (PS4), PS4 controller accessories and two gaming headsets. The held-out target is an *ARKARTECH Gaming Headset*. Every negative is a known, reviewed product, illustrating that the corrected lists contain no trivial zero-interaction decoys: a ranker must infer from four console-game reviews that an accessory such as a headset is a plausible next item.

B Errata: the Superseded Run

Table 10: Original (degenerate lists) versus corrected-list results, both with the GRU before the initialisation fix (Appendix C).

Quantity	Original lists	Corrected lists
Unknown negatives per list (of 19)	15.5	0
Popularity NDCG@5	0.801	0.109
GRU NDCG@5 (all users)	0.823	0.279
ARAG NDCG@5	0.453	0.424
α^*	0.1	0.9
H1 (ARAG + GRU − ARAG)	+0.349 "significant"	−0.013 n.s.
H2 (ARAG + GRU − GRU)	+0.002 n.s.	+0.174 significant
ARAG vs. Recency / Vanilla-RAG	n.s.	both significant

C Errata: the GRU Initialisation Bug

The first version of this report used PyTorch's default $\mathcal{N}(0, 1)$ initialisation for the GRU's item embeddings. With tied input and output embeddings, the initial logits are dot products of two such vectors, the softmax over 25,527 items is saturated (initial loss ≈ 30 versus $\ln 25{,}527 \approx 10$) and training stalls. The GRU reached only 0.293 validation and 0.279 test NDCG@5, which we attributed to short histories. When SASRec showed the same symptom, we re-initialised both models from $\mathcal{N}(0, 0.02^2)$, retrained both GRUs (same grid and seeds, epoch cap raised from 12 to 30), re-tuned α on validation and replayed the entire evaluation from the LLM cache at zero cost. The history-length analysis (Section 7.5) independently rules out the short-history explanation.

Table 11: Results before and after the initialisation fix.

Quantity	Before fix	After fix
GRU validation NDCG@5 (seed 0)	0.293	0.528
GRU test NDCG@5, all users	0.279	0.505
Text-aware GRU test NDCG@5, all users	0.461	0.530
GRU NDCG@5, 100 users (seed 0)	0.238	0.519
α^* (ARAG + GRU)	0.9	0.2
H1 (ARAG + GRU − ARAG)	−0.013 n.s.	+0.098 significant
H2 (ARAG + GRU − GRU)	+0.174 significant	+0.004 n.s.

At the same time we corrected the bootstrap for all-users comparisons, which had resampled a random 20,000-user subset (making those intervals about twice too wide and slightly off-centre); it now draws exact bootstrap means for all 94,762 users. No 100-user interval changed.

D Reproducibility

All code, configuration and results are in the project repository. The full pipeline is:

```
make setup                        # environment
make data lists index             # dataset, frozen lists, MiniLM + FAISS
make gru gru-text sasrec          # ID-only GRU, text-aware GRU, SASRec (grid + 3 seeds)
make eval                         # all systems (LLM calls disk-cached)
make report verify                # results/, and re-derive every metric from raw predictions
make extra                        # history length, SASRec contrasts, beyond-accuracy
python scripts/make_figures.py    # figures
python scripts/make_paper_tables.py   # the LaTeX tables of this report
```

A zero-cost replay of the full evaluation is `REPLAY_ONLY=1 OPENAI_API_KEY=replay-only-dummy make eval`; any LLM call missing from the cache raises an error instead of reaching the API. Ev-

ery table in Section 7 is generated from `results/metrics.json` and `results/extra_metrics.json` by `scripts/make_paper_tables.py`.